

Flood Detection and Scene Understanding using Deep Learning

Carissa Rego, Carly Carroll, and Simon Salaj

MSDS 610: Deep Learning

Code Repository: https://github.com/carlycarroll25/MSDS610_FinalProject

Introduction

Flooding is one of the most destructive natural disasters, causing widespread damage to infrastructure, ecosystems, and human life. Rapid and accurate assessment of flooded areas is critical for emergency response, resource allocation, and recovery planning. Traditional methods of flood detection often rely on manual inspection or satellite-based analysis, which can be both time consuming and limited in resolution.

This project aims to develop a deep learning model for post-disaster scene understanding using aerial imagery from the FloodNet dataset. Specifically, the task is to classify images as either flooded or non-flooded, enabling automated damage assessment. The project is inspired by the FloodNet dataset introduced by Rahnemoonfar et al. (2020), which provides high resolution UAV images designed for flood-related tasks.

The motivation behind this project is to leverage convolutional neural networks to extract features from aerial images and improve the efficiency and accuracy of flood detection systems.

Related Work

A relevant study in the domain of flood detection using deep learning is presented by Li et al. (2019), which explores the use of convolutional neural networks for flood mapping using remote sensing imagery. In their work, the authors apply deep learning techniques to satellite images to automatically identify flooded regions, demonstrating that CNN based models significantly outperform traditional thresholding and rule-based methods.

The study highlights the importance of spatial feature extraction, where convolutional layers are able to capture patterns such as water textures, reflections, and terrain changes that are indicative of flooding. Additionally, the authors emphasize the role of data preprocessing and augmentation, showing that model performance improves when trained on diverse environmental conditions and image variations.

One key contribution is the comparison of different deep learning architectures for flood detection, illustrating that deeper networks can capture more complex spatial relationships but may require more careful tuning to avoid overfitting. This aligns with the goals of the current project, which also includes a comparative analysis of architectures and training strategies.

While Li et al. (2019) focus on satellite imagery, this project extends similar deep learning principles to high resolution UAV imagery, which provides detailed local information. This allows for potentially improved classification performance and more precise scene understanding in post disaster environments.

Another relevant study is presented by Rahnemounfar and Sheppard (2017), who use deep convolutional neural networks to estimate flood levels from images, even in scenarios with limited real world labeled data. Their approach demonstrates that CNN based models can generalize well to flood related tasks by learning visual patterns such as water extent and surface texture. The study also highlights the effectiveness of leveraging pretrained models and synthetic data to improve performance when real datasets are limited. This supports the approach used in the current project, where transfer learning with pretrained ResNet architectures enables strong performance despite a relatively small dataset, reinforcing the value of pretrained feature extraction in flood detection tasks.

Data

The dataset used in this project is FloodNet, a high resolution UAV imagery dataset designed for post disaster flood scene understanding. The dataset contains images captured by UAVs, providing detailed spatial information suitable for tasks such as classification and segmentation.

The FloodNet dataset includes a total of 2,343 images, which are divided into training, validation, and test sets as follows:

- Training Set: 1,445 images
- Validation Set: 450 images
- Test Set: 448 images

Each image is paired with a corresponding segmentation mask of the same resolution, enabling both image level classification and pixel level analysis.

Additionally, the dataset contains multiple semantic classes, including flooded and non-flood buildings and roads, water, vegetation, and other scene elements. For this project, these classes will be simplified into a binary classification problem, where images are labeled as either flooded or non-flooded.

Preprocessing

Before training, all images were resized to 512×512 . This allowed for the experiments to be less computationally expensive, while still keeping enough visual detail for classification.

The segmentation masks were used to create binary labels, where flooded buildings and roads were treated as flooded classes. If a mask contained any pixels from those classes, the image was labeled as flooded. Otherwise, it was labeled as non-flooded.

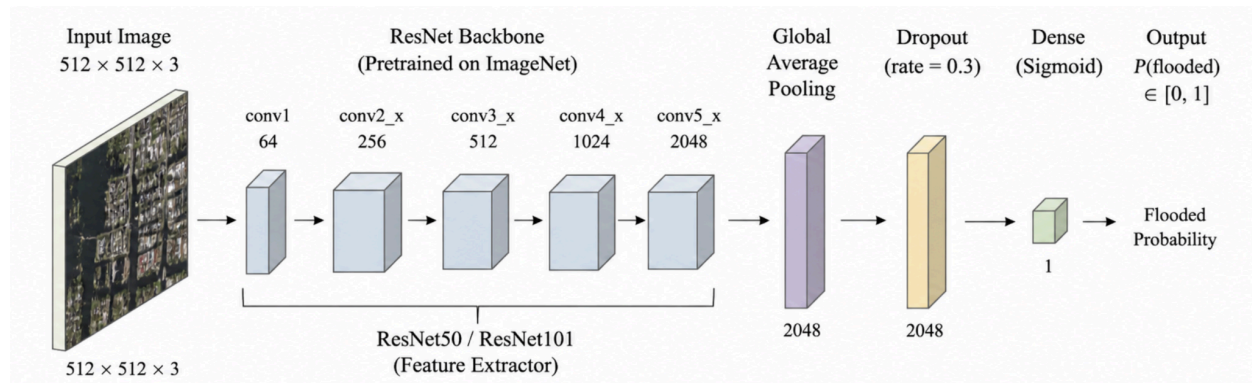
The preprocessing pipeline also included normalization based on the training set mean and standard deviation. A smoothing step was applied to the binary flood mask using a bilateral filter, dilation, and erosion to reduce noise before the final image label was created.

Methodology

Model Architecture

The project uses pretrained ResNet architectures because the task depends on recognizing spatial patterns in aerial imagery, such as water coverage, damaged surfaces, and flooded built environments. Instead of training a network from scratch, transfer learning made it possible to start from strong visual features learned on ImageNet and adapt them to the FloodNet classification task. This made the project more practical and better aligned with the size of the dataset.

Two architectures were used in the final implementation: ResNet50 and ResNet101. In both cases, the backbone model was used for feature extraction and then adapted for binary classification with a sigmoid output. This design fit the project well because the goal was not to predict multiple scene classes, but to decide whether flooding was present in a given image.



Input and Target Representation

The input space consists of RGB aerial images from FloodNet. The target space is binary, with each image labeled as either flooded or non-flooded. Rather than using the original multiclass segmentation output directly, the masks were converted into a simplified image-level target that reflects whether flooding is present in the scene.

This design makes the task more practical for rapid scene-level flood detection, where the goal is to determine whether a given aerial image contains flooding rather than to perform full pixel-level segmentation.

Loss Function

The model was trained using binary cross-entropy loss, commonly used for binary classification problems where the output represents a probability between 0 and 1.

$$L = -(y \log(p) + (1 - y) \log(1 - p))$$

where $y \in 0, 1$ is the true label and p is the predicted probability of the positive class.

This loss function measures the difference between the true labels and the predicted probabilities, penalizing confident but incorrect predictions more heavily. It is well suited for this task because the model outputs a probability, and the objective is to correctly classify images as flooded or non-flooded. In this setup, binary cross-entropy matched the structure of the task naturally and worked well for training both ResNet models.

Optimizer

The model used the Adam (Adaptive Moment Estimation) optimizer, which combines the advantages of momentum and adaptive learning rate methods.

Adam maintains exponential moving averages of both the first moment (mean) and the second moment (variance) of the gradients:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) \nabla L(\theta_{t-1}) \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) (\nabla L(\theta_{t-1}))^2 \end{aligned}$$

These estimates are then bias corrected:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Finally, the parameters are updated as:

$$\theta_t = \theta_{t-1} - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

where α is the learning rate and ϵ represents the error term.

Adam was used because the project relied on transfer learning and fine-tuning rather than training from scratch. In this setting, Adam provided a stable way to adapt pretrained ResNet models to the FloodNet dataset during both the initial training stage and the later fine-tuning stage. This was especially useful because the project involved deep architectures and limited computational resources.

Training Procedure

Training was done in two stages. First, the pretrained backbone was frozen and only the new classification head was trained. This allowed the model to adjust to the FloodNet task without changing the original pretrained features too early.

Following this, the backbone was partially unfrozen and the last 30 layers were fine-tuned. This gave the model a chance to adapt higher-level visual features to flood detection while keeping the earlier layers stable. Training used batch size 8, early stopping, and learning-rate reduction based on validation loss.

Experimental Component

The experimental part of the project is the comparison between ResNet50 and ResNet101. This was chosen to test whether a deeper model would improve performance on this task.

This comparison is meaningful because ResNet101 is more complex than ResNet50. If the deeper model performed better, that would suggest the task benefits from added depth. If not, it would show that the simpler model is already enough.

Metrics

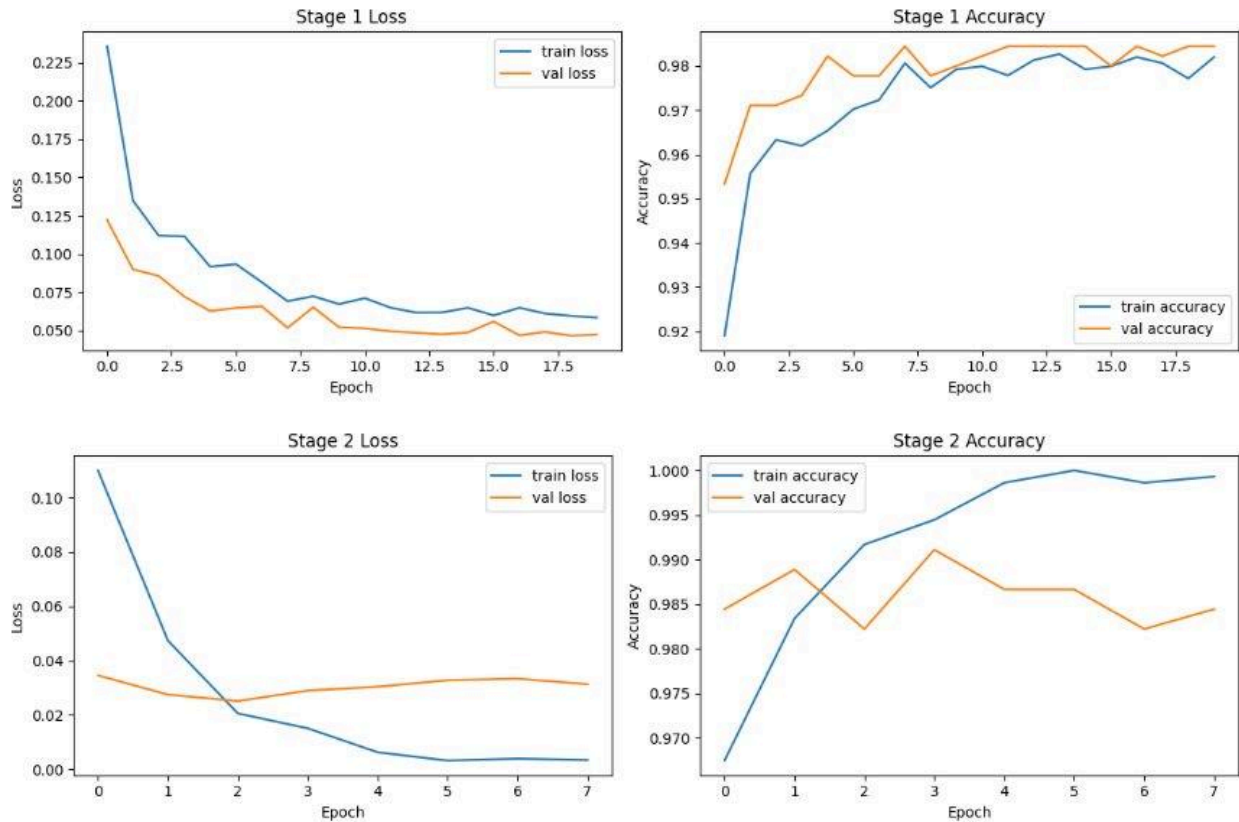
Model performance was evaluated using accuracy, precision, recall, F1-score, and confusion matrices. These metrics were chosen because overall accuracy alone would not fully capture model performance on a flood detection task, especially when missing flooded scenes is more serious than incorrectly flagging a non-flooded one.

F1-score and the confusion matrix were especially important for this project because they provided a clearer view of how well the models handled the flooded class. This was useful in interpreting the final results, since the models achieved very high overall performance but still made a small number of false-negative errors on flooded scenes.

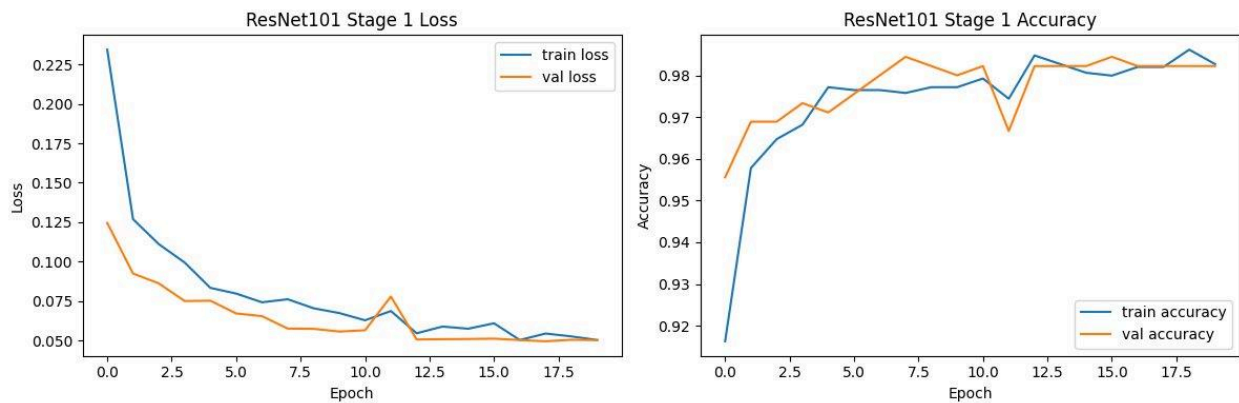
Results

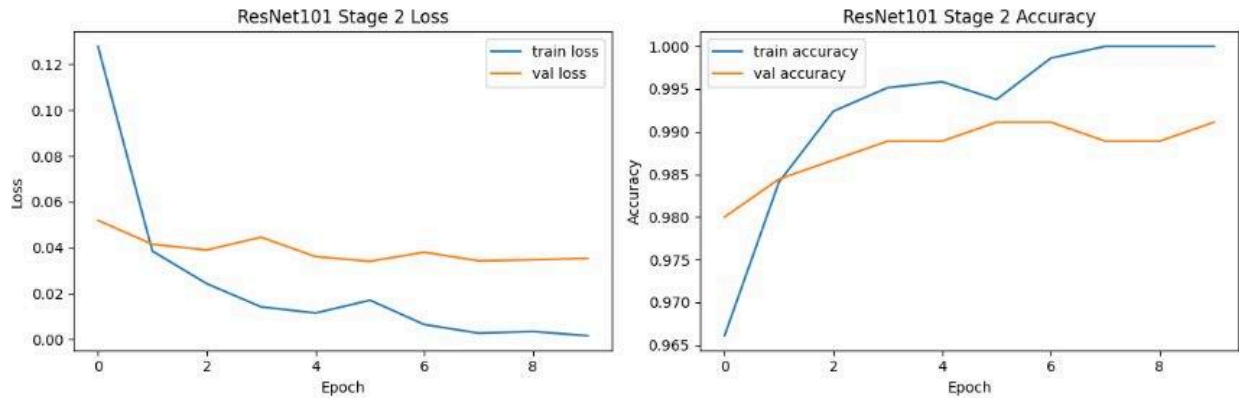
The performance of the ResNet50 and ResNet101 model is evaluated using training curves, test metrics, and confusion matrices. In addition to overall accuracy, differences in error patterns and prediction behavior are examined to provide a more complete understanding of model performance.

The training curves for ResNet50 show that during Stage 1, validation accuracy increases from approximately 0.92 to around 0.98 before stabilizing, while the loss steadily decreases, indicating effective learning with the frozen backbone. In Stage 2, fine-tuning further improves performance, with training accuracy approaching 1.0 and validation loss reaching a minimum of approximately 0.025. Early stopping restores the best weights based on validation loss, helping to prevent overfitting. These curves demonstrate stable convergence and effective use of the two stage training strategy.



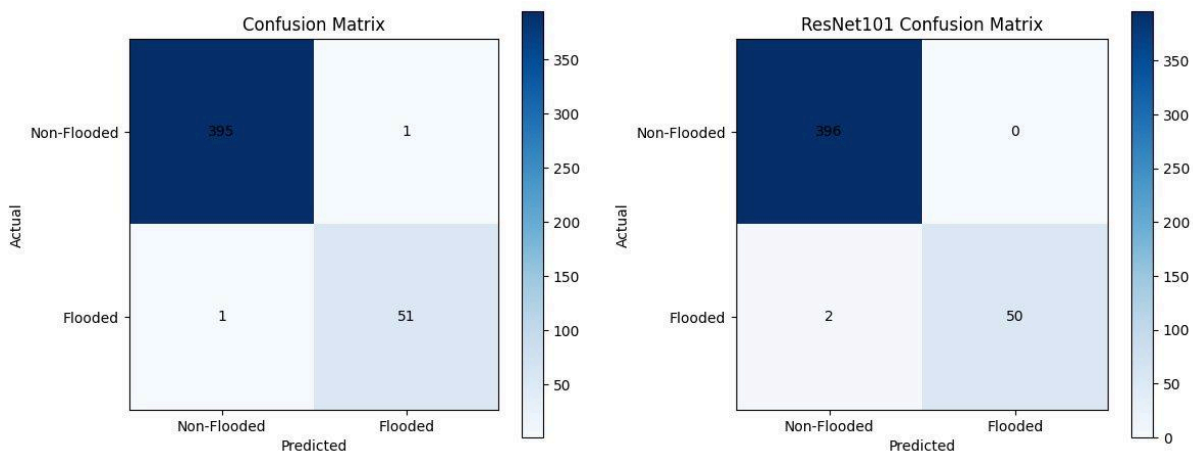
The training curves for ResNet101 follow a similar pattern, with validation accuracy increasing rapidly during Stage 1 and stabilizing at a high level. Compared to ResNet50, the Stage 2 curves appear slightly more stable, with less divergence between training and validation performance, which is consistent with the increased capacity of the deeper network. The best validation accuracy reaches approximately 0.991, with training continuing for around 30 epochs. Overall, both models exhibit strong and stable learning behavior across both training stages.





On the test set, both models achieve identical accuracy of 99.55%, indicating that increasing model depth does not improve overall classification performance for this task. ResNet50 achieves an F1 score of 0.9808m with precision and recall both equal to 0.9808, reflecting balanced performance across classes. In contrast, ResNet101 achieves a slightly lower F1 score of 0.9804, with precision of 1.0000 and a recall of 0.9615, indicating that the model produces no false positives but misses a small number of flooded cases.

A side by side comparison of the confusion matrices highlights this difference in error behavior. ResNet50 produces symmetric error, with 1 false positive and 1 false negative, while ResNet101 exhibits a more conservative pattern, with 0 false positives and 2 false negatives. This trade off shows that while both models perform equally well in terms of accuracy, they differ in how they distribute prediction errors.



Overall, the results suggest that the task is sufficiently separable and that ResNet50 already has enough capacity to capture the relevant spatial features present in the data. Increasing model depth through ResNet101 does not provide a meaningful improvement in performance, but instead shifts the model toward a more precision-focused prediction strategy. As a result, the

choice between models depends on the application: ResNet50 is better suited for balance screening scenarios, while ResNet101 is preferable in settings where false positives are more costly.

These findings also reinforce several broader observations. Transfer learning proves highly effective, as both models achieved strong performance with relatively limited fine tuning. The two stage training approach contributes to stable optimization and improved convergence. Finally, differences in error behavior, rather than overall accuracy, emerge as the most important factor when comparing model performance in this task.

Challenges

The most significant challenges encountered in this project has been managing computational constraints while scaling the model and preprocessing pipeline. Increasing image resolution to our desired size of 512 x 512 and experimenting with deeper architectures including ResNet 101 led to noticeably longer training times. As a result, it was necessary to find a balance between model complexity and computational efficiency. In particular, considerations of the trade offs between achieving potentially higher performance with more complex models had to be made, and the increased computational cost and runtime associated with them. This required making practical decisions about image size, model depth, and preprocessing steps to ensure that the model remained feasible to train while still producing strong results.

Another challenge encountered was converting the original multi class segmentation masks into binary image level labels. The FloodNet dataset was designed for pixel level segmentation, so a choice had to be made about which images would be labeled as flooded or not flooded. So images with even very small flooded regions were still classified as flooded even if the majority of the pixels were not flooded.

The final challenge faced was class imbalance during training. Accuracy alone can bias a model towards predicting the majority class in an imbalanced dataset, and to help mitigate this additional metrics (precision, recall, F1-score, and confusion matrices) were used to get a more complete picture of the models performance. Both models still misclassified a small number of flooded images. This is most likely because of cases where there may have been extremely minor flooding or too weak of a visual signal for either model to capture.

An additional limitation relates to how the FloodNet dataset splits were constructed. An analysis of image filenames revealed that 8 scene groups, meaning images captured from the same UAV flight, appear in both the training and test sets. Since images from the same flight share similar visual characteristics such as lighting, terrain, and camera angle, this overlap may make the test set easier for the model than truly unseen data would be. This is a dataset-level limitation rather

than an issue with the model or training procedure, but it is worth noting when interpreting the high test accuracy.

Reflection

The project exceeded expectations for each of the goals set. The base goal was to train a model that achieved at least 80% accuracy on the test set, the target goal was 85% accuracy with balanced precision and recall, and the stretch goal was to get close to 90% and clearly identify which setup performed the best. The final results exceeded all three of these goals, ResNet50 and ResNet101 both achieved a test accuracy of 99% with strong scores for precision, recall, and F1-score.

The models behaved as expected to an extent, although the identical performance of both ResNet50 and ResNet101 wasn't completely foreseen. The assumption was that ResNet101 would perform noticeably better than ResNet50 due to the more complex architecture. However, both models misclassified only two images, ResNet50 misclassified one flooded image and one non flooded image, and ResNet101 misclassified two flooded images as non flooded. These results confirmed that the classification task did not require the more complex model of ResNet101, and that ResNet50 is the more practical and computationally effective model choice for this task.

One pivot that was made was the choice of architectures to use when comparing performance. Initially ResNet18 and ResNet50 were going to be compared, but that was pivoted to comparing ResNet50 and ResNet101. Another pivot was the decision to not use data augmentation or pixel level segmentations as extensions of the project, these were discussed initially as potential extensions only, and overall were not used due to the high performance achieved by both models without implementing either.

With more time data augmentation could be implemented to improve the robustness of the model when seeing environmental variation not well represented in the training data. Adjusting the classification threshold and applying class weights during training could potentially lower the false negatives predicted. To expand on a broader scale, utilizing pixel level segmentation opposed to image level classification would make the model results more applicable to real world applications such as emergency response scenarios.

The biggest takeaway from this project is that larger models and more complex architecture does not always equate to better performance and the nature of the task is very important when determining the model that is the best fit. Transfer learning was a highly effective approach in this project and produced very successful results. Not only using accuracy, but also precision, recall and F-1 scores, to evaluate both models was crucial in understanding the full picture when it came to model performance.

References

- Li, Y., Martinis, S., & Wieland, M. (2019). Urban flood mapping with an active self-learning convolutional neural network based on TerraSAR-X intensity and interferometric coherence. *ISPRS Journal of Photogrammetry and Remote Sensing*, 152, 178-191.
- Rahnemoonfar, M., Chowdhury, T., Sarkar, A., Varshney, D., Yari, M., & Murphy, R. R. (2021). Floodnet: A high resolution aerial imagery dataset for post flood scene understanding. *IEEE Access*, 9, 89644-89654.